



Franklin Templeton: Aptos Money Market Fund

Security Assessment

October 11, 2024

Prepared for:

Franklin Templeton Digital Assets

Prepared by: **Anish Naik and Jaime Iglesias**

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review projects, supporting client organizations in the technology, defense, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, and Zoom.

Trail of Bits also operates a center of excellence with regard to blockchain security. Notable projects include audits of Algorand, Bitcoin SV, Chainlink, Compound, Ethereum 2.0, MakerDAO, Matic, Uniswap, Web3, and Zcash.

To keep up to date with our latest news and announcements, please follow [@trailofbits](#) on Twitter and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact>, or email us at info@trailofbits.com.

Trail of Bits, Inc.

497 Carroll St., Space 71, Seventh Floor
Brooklyn, NY 11215

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2024 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

This report is considered by Trail of Bits to be business confidential information; it is licensed to Franklin Templeton under the terms of the project statement of work and intended solely for internal use by Franklin Templeton. Material within this report may not be reproduced or distributed in part or in whole without the express written permission of Trail of Bits.

The sole canonical source for Trail of Bits publications, if published, is the [Trail of Bits Publications page](#). Reports accessed through any source other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

All activities undertaken by Trail of Bits in association with this project were performed in accordance with a statement of work and agreed upon project plan.

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test the controls and security properties of software. These techniques augment our manual security review work, but each has its limitations: for example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. Their use is also limited by the time and resource constraints of a project.

Table of Contents

About Trail of Bits	1
Notices and Remarks	2
Table of Contents	3
Project Summary	5
Executive Summary	6
Project Goals	9
Project Targets	10
Project Coverage	11
Codebase Maturity Evaluation	14
Summary of Findings	16
Detailed Findings	18
1. Users can request a share transfer when self-service is disabled	18
2. Full liquidation requests will fail to execute	20
3. A malicious shareholder can launch a DoS attack	22
4. Shareholders can stop the admin from removing them as a shareholder	25
5. Insufficient event generation	27
6. Frozen shareholders can be distributed dividends	28
7. Insufficient checks during request settlement	30
8. A high number of pending requests could lead to a temporary denial of service	34
9. Pending requests are not settled in the order in which they were created	36
10. The order in which the admin inputs the accounts to the settlement function can alter the outcome of the settlement	38
11. Creating requests just before the end of day allows shareholders to instantly settle them	40
12. The end_of_day function is broken	43
13. Multisig integrity checks are not performed on addresses with PERMISSION_LEVEL_ALL role	45
14. The recover_account and recover_assets functions do not check whether the source and destination addresses are the same	48
15. Account recovery can be prevented by creating a pending transaction	50
16. Beware of changing token decimals	52
17. The addition of negative yield, asset recovery, and share adjustment can also trigger TOB-FT-APTOS-7	54
18. There is no association between multisig addresses and their underlying	

configuration	55
A. Vulnerability Categories	59
B. Code Maturity Categories	61
C. Non-Security-Related Findings	63
D. Fix Review Results	65
Detailed Fix Review Results	66
E. Fix Review Status Categories	69

Project Summary

Contact Information

The following project manager was associated with this project:

Sam Greenup, Project Manager
sam.greenup@trailofbits.com

The following engineering director was associated with this project:

Josselin Feist, Engineering Director, Blockchain
josselin.feist@trailofbits.com

The following consultants were associated with this project:

Anish Naik Consultant
anish.naik@trailofbits.com

Jaime Iglesias, Consultant
jaime.iglesias@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
August 1, 2024	Pre-project kickoff call
August 12, 2024	Status update meeting #1
August 15, 2024	Delivery of report draft
August 15, 2024	Report readout meeting
September 5, 2024	Delivery of report draft with fix review addendum
October 7, 2024	Delivery of report draft with best-effort review
October 11, 2024	Delivery of comprehensive report with best-effort review

Executive Summary

Engagement Overview

Franklin Templeton engaged Trail of Bits to review the security of its Aptos Money Market Fund (MMF). The MMF constitutes three total modules that are responsible for shareholder onboarding, creation of shareholder requests, settlement of those requests, distribution of dividends, and the management of administrative permissions and roles.

A team of two consultants conducted the review from August 5 to August 14, 2024, for a total of three engineer-weeks of effort. Our testing efforts focused on identifying access control bypasses, whether it is possible to steal funds, transaction ordering risks, denial-of-service (DoS) attacks, data validation concerns, insufficient error handling, and violations to any critical system invariants. With full access to source code and documentation, we performed a manual review of the codebase. Note that the MMF heavily relies on off-chain components, which were considered out of scope.

Observations and Impact

We identified one core concern that led to a majority of the critical issues identified during this audit: the system has insufficient testing.

First, there is a significant lack of negative unit tests. Negative unit testing aids in validating how the system behaves when bad user input is provided. For example, **TOB-FT-APTOS-1** allows users to use the self-service capability for share transfers even when self-service is disabled. A negative unit test that checks that `request_self_service_share_transfer` should abort if self-service is disabled would have prevented this issue. Similarly, a test to make sure that a shareholder cannot cancel a request that they did not make would have prevented **TOB-FT-APTOS-3**. Part of the reason why there is insufficient negative unit testing is because the system suffers from insufficient error handling. In many cases, the system should throw an error if some index does not exist in a vector or a key is not in the smart table. Improving error handling allows the developer to quickly generate negative unit tests to trigger each case where an error should be thrown. By ignoring errors, new code paths are created for exploitation.

More generally, the system would benefit from richer unit testing. **TOB-FT-APTOS-9** is an example of an issue that can be triggered only in specific circumstances. The test cases for settlement processing always provide a large initial share balance for all the shareholders and no request can cause the user to have insufficient funds to complete the request. Thus, the bug cannot be triggered since the scarcity of a user's share balance is never a core consideration in the test. Similarly, **TOB-FT-APTOS-2** showcases that there is insufficient branch coverage. As in the previous example, the settlement processing tests do not have enough variety of shareholder operations or sufficient post-condition testing to validate the system state after transaction execution.

It is important to note that the concerns raised in this audit mostly pertained to the fund token module. The ERC20 wrapper and access control module were of higher quality and had sufficient unit testing.

Recommendations

Based on the codebase maturity evaluation and findings identified during the security review, Trail of Bits recommends that Franklin Templeton take the following steps prior to deployment:

- **Remediate the findings disclosed in this report.** These findings should be addressed as part of a direct remediation or as part of any refactor that may occur when addressing other recommendations.
- **Improve unit testing.** The system suffers from insufficient unit testing. Improving error handling, adding negative unit tests, enriching existing unit testing, and improving post-condition checks will significantly improve the security posture of the system.
- **Identify and document any risks related to timing.** The delay between shareholder requests and settlement processing may lead to unexpected behavior (e.g. price changes or updates to shareholder state). It is beneficial to identify what edge cases can be mitigated and what edge cases are accepted as risks.
- **Integrate the Move Prover.** The built-in Move Prover is a powerful testing methodology to validate the system's invariants. More generally, a good first step in this direction is to document the system's invariants. Identifying and documenting them aids in validating system behavior.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

<i>Severity</i>	<i>Count</i>
High	4
Medium	4
Low	1
Informational	9
Undetermined	0

CATEGORY BREAKDOWN

<i>Category</i>	<i>Count</i>
Auditing and Logging	1
Data Validation	9
Denial of Service	2
Timing	1
Undefined Behavior	5

Project Goals

The engagement was scoped to provide a security assessment of Franklin Templeton's Aptos MMF. Specifically, we sought to answer the following non-exhaustive list of questions:

- Can frozen shareholders interact with the system in unexpected ways?
- Can users steal from the fund module?
- Are user requests created and processed correctly?
- Are access controls implemented correctly?
- Are there any potential DoS vectors?
- Can user requests be settled in unexpected ways?
- Is the access control module implemented correctly?
- Does the ERC20 wrapper module lead to any violations of the fungible asset (FA) standard's invariants?

Project Targets

The engagement involved a review and testing of the targets listed below.

Comprehensive Review

funds

Filepath **aptos-tokenized-fund/funds**

SHA-256 Hash
b92698e755a1c756535ad34ae1ed84dea3025ef55694a41b94af176e50c0025e

Type Aptos Move

Platform Aptos

multisig

Filepath **aptos-tokenized-fund/multisig**

SHA-256 Hash
b92698e755a1c756535ad34ae1ed84dea3025ef55694a41b94af176e50c0025e

Type Aptos Move

Platform Aptos

Best-Effort Review

funds

Filepath **aptos-tokenized-fund/funds**

SHA-256 Hash
af7bf35742616282ae2a864ba549412f9f6e66ebf39a9ef518cc55e6cd19b4d9

Type Aptos Move

Platform Aptos

multisig

Filepath **aptos-tokenized-fund/multisig**

SHA-256 Hash
af7bf35742616282ae2a864ba549412f9f6e66ebf39a9ef518cc55e6cd19b4d9

Type Aptos Move

Platform Aptos

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- **Fund token module.** The fund token module is responsible for handling shareholder actions (e.g., cash purchases, liquidations, or share transfers) and administrative actions (e.g., self-service enablement, settlement of transactions). We performed a manual review of this module and investigated the following:
 - We reviewed the authorization measures placed on each function to ensure that access controls cannot be bypassed. This investigation did not lead to any findings.
 - We reviewed the data validation within the module to validate whether the system handled bad user inputs correctly. This led to the discovery of **TOB-FT-APTOS-1**, which highlights that shareholders can use the self-service capability to perform share transfers even when self-service is disabled. Additionally, **TOB-FT-APTOS-3** highlights that a malicious shareholder can launch a DoS attack by canceling all other shareholder requests.
 - We reviewed whether the module was vulnerable to transaction ordering risks that could cause undefined behavior. This led to the discovery of **TOB-FT-APTOS-4**, which highlights that a malicious shareholder can launch a front-running attack to prevent themselves from being removed as a shareholder. Additionally, **TOB-FT-APTOS-11** highlights that a shareholder can cause their transactions to be immediately processed by front-running the invocation to the end-of-day function.
 - We reviewed the end-to-end flow of creating requests (with or without self-service) and the settlement of these requests for general correctness. This led to the discovery of **TOB-FT-APTOS-9**, which highlights that processing shareholder requests in reverse order may prevent requests from being successfully settled. Additionally, we discovered that the checks performed during the settlement of requests are insufficient and could lead to unwanted behavior or DoS (**TOB-FT-APTOS-7**). Finally, we noticed that one of the main administrative functions is broken (**TOB-FT-APTOS-12**).
 - Regarding the settlement of requests, we also identified another potential DoS vector in **TOB-FT-APTOS-8** and a potential point of degraded user experience in **TOB-FT-APTOS-10**.

- We reviewed the module's interactions with the ERC20 wrapper and access control modules for general correctness. This investigation did not lead to any findings.
- We reviewed whether it is possible for a shareholder to transfer shares to a non-shareholder. This investigation did not lead to any findings.
- We reviewed the arithmetic operations within the module for risks related to arithmetic overflow and rounding errors. This investigation did not lead to any findings.
- **ERC20 wrapper module.** The ERC20 wrapper module is an abstraction on top of the FA standard of Aptos and represents user shares of the money market fund. We performed a manual review of this module and investigated the following:
 - We reviewed the authorization measures placed on each function to assess whether access controls can be bypassed. This investigation did not lead to any findings.
 - We reviewed the module's interactions with the FA module and reviewed whether these interactions lead to the violation of any FA standard invariants. This led to the discovery of **TOB-FT-APTOS-7**, which highlights that dividends can be distributed to shareholders that have their FA stores frozen.
- **Access control module.** The access control module is a self-contained resource account that holds an access-control list for allowlisted administrators. The module allows for the generation of a shared signer that can be used for authorization in client modules (e.g., the fund token module). We performed a manual review of this module and investigated the following:
 - We reviewed whether it was possible for a malicious user to bypass access controls and become the module's administrator. This investigation did not lead to any findings.
 - We reviewed the module for general correctness and whether an account with a lower permission level can impersonate an account with a higher permission level. This investigation did not lead to any findings.

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- **Off-chain components.** The money market fund heavily relies on off-chain components. These off-chain components are responsible for cash settlements, execution of administrative actions, and distribution of dividends. These components were considered out of scope. Our review assumed that these components function as expected.
- **Best-effort review.** After the original engagement (and its subsequent fix review), the client requested a review of a few additional features. These included upgradeability, additional limitations to the access control module, and new fund module features such as negative yield or asset recovery.

We reviewed these changes on a best-effort basis for two engineer-days and uncovered six issues ([TOB-FT-APTOS-13](#), [TOB-FT-APTOS-14](#), [TOB-FT-APTOS-15](#), [TOB-FT-APTOS-16](#), [TOB-FT-APTOS-17](#), and [TOB-FT-APTOS-18](#)). These issues indicate that the testing suite is still immature, although we should highlight that it has improved since the original engagement. We recommend that the client address these issues and continue to improve the testing suite with additional positive and negative cases, along with integration tests that include features such as module upgrades and deployments.

Finally, given that a small amount of code changes introduced several issues, we recommend that the client take a more systematic approach when shipping new features, by first defining and documenting system invariants and then driving testing and development through them. This will help ensure that more secure code is shipped in the future.

Codebase Maturity Evaluation

Trail of Bits uses a traffic-light protocol to provide each client with a clear understanding of the areas in which its codebase is mature, immature, or underdeveloped. Deficiencies identified here often stem from root causes within the software development life cycle that should be addressed through standardization measures (e.g., the use of common libraries, functions, or frameworks) or training and awareness programs.

Category	Summary	Result
Arithmetic	The arithmetic within the modules is of generally low complexity. We identified no issues in relation to arithmetic overflows, division by zero, or precision loss.	Satisfactory
Auditing	Most state-changing functions emit events. However, we identified some functions that would benefit from event emission (TOB-FT-APTOS-5). Additionally, the system logs all shareholder requests and settlements so that they can be leveraged by off-chain components for monitoring.	Moderate
Authentication / Access Controls	We did not identify any vectors that would allow for access control bypasses. However, due to insufficient authorization, we identified one instance that would allow a malicious shareholder to cancel all pending transactions of all the other shareholders (TOB-FT-APTOS-3).	Moderate
Complexity Management	The various process flows and interactions between modules are generally easy to reason through. However, we identified a number of instances where redundant data is stored on-chain. Additionally, some functions can be removed or simplified. Recommendations for reducing complexity can be found in appendix C .	Moderate
Cryptography and Key Management	We identified no usages of cryptographic primitives or functions.	Not Applicable
Decentralization	The system is centralized in nature. Most user operations require authorization from an allowlisted administrator to be executed. A large amount of trust must be placed in	Weak

	the allowlisted administrators since they are responsible for updating prices, settling requests, and distributing dividends. If self-service is disabled, users cannot exit the system without authorization.	
Documentation	The provided documentation is generally high-quality. The documentation highlights the system actors and process flows. The inline documentation is sufficient. However, additional documentation pertaining to running tests and deploying the modules would be beneficial, as we ran into several problems while trying to compile the modules and to run the tests that needed troubleshooting with the client.	Satisfactory
Testing and Verification	We identified a large number of issues that should have been caught through unit testing. For example, TOB-FT-APTOS-1, TOB-FT-APTOS-2, and TOB-FT-APTOS-3 all showcase shortcomings in negative unit testing (handling bad user inputs), and TOB-FT-APTOS-12 highlights some functions that have no tests at all. Having better error handling would also aid in this endeavor. We also found deeper-rooted issues, such as TOB-FT-APTOS-9, that could have been caught with richer unit tests that can enter specific edge cases.	Weak
Transaction Ordering	We identified a few transaction-ordering risks. The most interesting of these is the ability to have a request immediately settled, which may cause undefined behavior for off-chain cash settlements (TOB-FT-APTOS-11). More generally, the separation of request creation and request settlement can create unexpected timing-related issues within the system (TOB-FT-APTOS-7, TOB-FT-APTOS-10).	Moderate

Summary of Findings

The table below summarizes the findings of the review, including type and severity details.

ID	Title	Type	Severity
1	Users can request a share transfer when self-service is disabled	Data Validation	Medium
2	Full liquidation requests will fail to execute	Data Validation	Medium
3	A malicious shareholder can launch a DoS attack	Denial of Service	High
4	Shareholders can stop the admin from removing them as a shareholder	Timing	Informational
5	Insufficient event generation	Auditing and Logging	Informational
6	Frozen shareholders can be distributed dividends	Data Validation	Informational
7	Insufficient checks during request settlement	Data Validation	High
8	A high number of pending requests could lead to a temporary denial of service	Denial of Service	Informational
9	Pending requests are not settled in the order in which they were created	Data Validation	High
10	The order in which the admin inputs the accounts to the settlement function can alter the outcome of the settlement	Undefined Behavior	Informational
11	Creating requests just before the end of day allows shareholders to instantly settle them	Undefined Behavior	Informational
12	The end_of_day function is broken	Data Validation	High

13	Multisig integrity checks are not performed on addresses with PERMISSION_LEVEL_ALL role	Data Validation	Medium
14	The recover_account and recover_assets functions do not check whether the source and destination addresses are the same	Data Validation	Low
15	Account recovery can be prevented by creating a pending transaction	Data Validation	Informational
16	Beware of changing token decimals	Undefined Behavior	Informational
17	The addition of negative yield, asset recovery, and share adjustment can also trigger TOB-FT-APTOS-7	Undefined Behavior	Informational
18	There is no association between multisig addresses and their underlying configuration	Undefined Behavior	Medium

Detailed Findings

1. Users can request a share transfer when self-service is disabled

Severity: Medium

Difficulty: Low

Type: Data Validation

Finding ID: TOB-FT-APTOS-1

Target: funds/sources/fund_token.move

Description

Users can transfer FA units to other shareholders without authorization from an allowlisted administrator.

Shareholder operations, such as share transfers, liquidations, or cash purchases, must be performed by an allowlisted administrator. If self-service is enabled, however, shareholders can perform these actions without authorization. A self-service share transfer can be initiated by calling the `request_self_service_share_transfer` function (figure 1.1).

```
689     public entry fun request_self_service_share_transfer(  
690         caller: &signer,  
691         destination: address,  
692         shares: u64,  
693         memo: String,  
694     ) acquires ManagedShareHolders, FundManagement {  
695         only_when_initialized();  
696         only_when_account_exists(signer::address_of(caller));  
697         only_when_account_exists(destination);  
698         only_when_account_not_frozen(signer::address_of(caller));  
699         only_when_account_not_frozen(destination);  
700         only_when_shareholder_has_shares(signer::address_of(caller));  
701         only_higher_than_zero(shares);  
702         only_when_shareholder_has_enough_shares(signer::address_of(caller),  
shares);  
703  
704         create_pending_transaction(signer::address_of(caller), destination,  
TX_TYPE_SHARE_TRANSFER, shares, false, memo);  
705     }
```

Figure 1.1: funds/sources/fund_token.move#L689–L705

However, notice that the function does not check whether self-service is enabled. Thus, a shareholder can transfer shares to another shareholder even when self-service is disabled.

Additionally, it is important to note that the function call to `create_pending_transaction` (line 704 in figure 1.1) incorrectly passes a value of `false` for the `self_service` input parameter (line 1052 in figure 1.2).

```
1047 fun create_pending_transaction(  
1048     account: address,  
1049     destination: address,  
1050     tx_type: u64,  
1051     amount: u64,  
1052     self_service: bool,  
1053     memo: String,  
1054 ) acquires ManagedShareHolders {
```

Figure 1.2: funds/sources/fund_token.move#L1047–L1054

Exploit Scenario

Eve is able to transfer shares to Alice while self-service is disabled and without the authorization by an allowlisted administrator.

Recommendations

Short term, call the `only_when_self_service_enabled` function in `request_self_service_share_transfer` to ensure that it reverts if self-service is disabled. Additionally, update the call to the `create_pending_transaction` function (figure 1.3):

```
704 create_pending_transaction(signer::address_of(caller), destination,  
TX_TYPE_SHARE_TRANSFER, shares, true, memo);
```

Figure 1.3: The call to `create_pending_transaction` correctly highlights that self-service is enabled.

Long term, improve unit testing. Specifically, adding negative unit tests to validate how functions behave with bad user inputs would aid in identifying additional edge cases like this.

2. Full liquidation requests will fail to execute

Severity: Medium

Difficulty: Low

Type: Data Validation

Finding ID: TOB-FT-APTOS-2

Target: funds/sources/fund_token.move

Description

Full liquidation requests by shareholders do not lead to the burning of any shares and may allow users to steal funds off-chain.

A full liquidation request is initiated by calling the `request_full_liquidation` function. This will result in the creation of a pending transaction that will be later settled by an administrator (figure 2.1).

```
514 public entry fun request_full_liquidation(  
515     caller: &signer,  
516     account: address,  
517     memo: String,  
518 ) acquires ManagedShareHolders, FundManagement {  
519     only_whitelisted(caller);  
520     only_when_initialized();  
521     only_when_account_exists(account);  
522     only_when_account_not_frozen(account);  
523     only_when_shareholder_has_shares(account);  
524  
525     create_pending_transaction(account, @0x0, TX_TYPE_CASH_LIQUIDATION, 0,  
false, memo);  
526 }
```

Figure 2.1: funds/sources/fund_token.move#L514–L526

However, notice that the transaction type, `TX_TYPE_CASH_LIQUIDATION`, and the number of shares to be burned, zero, are incorrect (line 525 in figure 2.1). Thus, when the pending transaction is settled, no shares are burned (line 1157 in figure 2.2).

```
1156 if (tx.tx_type == TX_TYPE_CASH_LIQUIDATION) {  
1157     if (tx.amount > 0) {  
1158         shares = calculate_shares(tx.amount, price,  
erc20_wrapper::decimals(fund.symbol));  
1159         erc20_wrapper::burn(resource_signer, account, shares, fund.symbol);  
1160     }  
1161 }
```

Figure 2.2: funds/sources/fund_token.move#L1156–L1161

Note that the severity of this issue may be higher if the off-chain components that settle liquidations fail to check if the requisite amount of shares were burned. If not, a shareholder can continue requesting full liquidations without burning any tokens but receiving cash off-chain.

Exploit Scenario

Eve, a shareholder, requests a full liquidation. When the request is settled, no tokens are burned.

Recommendations

Short term, update the `request_full_liquidation` function as follows:

```
525         create_pending_transaction(account, @0x0, TX_TYPE_FULL_LIQUIDATION, 0,  
false, memo);  
526     }
```

Figure 2.3: The call to `create_pending_transaction` correctly sets the transaction type to `TX_TYPE_FULL_LIQUIDATION`

Long term, improve unit testing. Specifically, sufficient post-condition testing must be implemented to ensure that all state-changes made by a function call are validated.

3. A malicious shareholder can launch a DoS attack

Severity: High

Difficulty: Low

Type: Denial of Service

Finding ID: TOB-FT-APTOS-3

Target: funds/sources/fund_token.move

Description

A malicious shareholder can launch a denial-of-service (DoS) by canceling all other shareholder requests.

A shareholder who wishes to cancel a self-service request can use the `cancel_self_service_request` function (figure 3.1). The function should allow a shareholder to cancel requests that were (1) created when self-service was enabled and (2) made by the shareholder.

```
710 public entry fun cancel_self_service_request(  
711     caller: &signer,  
712     request_id: u64,  
713     memo: String,  
714 ) acquires ManagedShareHolders {  
715     only_when_initialized();  
716     only_when_account_exists(signer::address_of(caller));  
717  
718     let managed = borrow_global_mut<ManagedShareHolders>(@ft_funds);  
719     let shareholders = &mut managed.shareholders;  
720     let shareholder = smart_table::borrow_mut(shareholders,  
signer::address_of(caller));  
721  
722     // remove transaction from the list of pending transactions  
723     if (smart_table::contains(&managed.pending_transactions, request_id)) {  
724         smart_table::remove(&mut managed.pending_transactions, request_id);  
725     };  
726  
727     // remove transaction request id from the vector of request ids (keys)  
728     let (ok, index) = vector::index_of(&shareholder.request_ids,  
&request_id);  
729     if (ok) {  
730         let counter = shareholder.counter;  
731         vector::remove(&mut shareholder.request_ids, index);  
732         shareholder.counter = counter - 1;  
733     };  
734  
735     // remove from the vector used to track of accounts with pending  
transactions
```

```

736         if (vector::length(&shareholder.request_ids) == 0) {
737             let (ok, index) =
vector::index_of(&managed.shareholder_accounts_with_pending_transactions,
&signer::address_of(caller));
738             if (ok) {
739                 let counter =
managed.shareholder_accounts_with_pending_transactions_counter;
740                 vector::remove(&mut
managed.shareholder_accounts_with_pending_transactions, index);
741                 managed.shareholder_accounts_with_pending_transactions_counter =
counter - 1;
742             };
743         };
[... ]
751     }

```

Figure 3.1: *funds/sources/fund_token.move#L710-L751*

However, neither constraint highlighted above is satisfied. Note that the function does not check whether the request that is trying to be canceled was created using the self-service capability. Thus, requests made with or without self-service can now be canceled.

More importantly, there is nothing to guarantee that the shareholder is canceling a request that they created. As shown by lines 728-729 in figure 3.1, no error is thrown if the request ID is not found in the vector of requests created by the shareholder. Thus, the deletion of the request in lines 723-725 will succeed. In fact, an error should be thrown if the request ID cannot be found at all. Moreover, lines 737-738 should also throw an error if the user cannot be found in the vector (note, however, that this check can be bypassed).

Exploit Scenario

Eve, a malicious shareholder, launches a DoS attack by continuously deleting all requests made by other shareholders. This would prevent the settlement of any future shareholder transaction.

Recommendations

Short term, update the `cancel_self_service_request` as following:

1. Throw an error if the shareholder does not own the request ID that they are trying to delete.
2. Throw an error if the request ID cannot be found in the `managed.pending_transactions` smart table.
3. Ensure that the request was made while self-service was enabled. This can be checked by ensuring that the `pending_transaction.self_service` variable is `true`.

4. Throw an error if the shareholder account is not in the `managed.shareholder_accounts_with_pending_transactions` vector.

Long term, improve negative unit testing to validate how functions behave with bad user inputs; this will aid in identifying additional edge cases like this. Additionally, improve error handling throughout the system. We identified many instances where functions will continue execution when in fact an error should be thrown. Note that thorough error handling directly aids in negative unit testing.

4. Shareholders can stop the admin from removing them as a shareholder

Severity: Informational

Difficulty: Medium

Type: Timing

Finding ID: TOB-FT-APTOS-4

Target: funds/sources/fund_token.move

Description

To remove a shareholder, the shareholder must not have any pending transactions that require settlement (line 799 in figure 4.1).

```
787 public entry fun remove_shareholder(  
788     caller: &signer,  
789     account: address,  
790 ) acquires ManagedShareHolders, FundManagement {  
791     only_whitelisted(caller);  
792     only_when_initialized();  
793     only_when_account_exists(account);  
794     only_when_shareholder_has_no_shares(account);  
795  
796     let managed = borrow_global_mut<ManagedShareHolders>(@ft_funds);  
797     let shareholder = smart_table::borrow_mut(&mut managed.shareholders,  
account);  
798  
799     only_when_shareholder_has_no_pending_transactions(shareholder);  
[...]  
825 }
```

Figure 4.1: funds/sources/fund_token.move#L787–L825

A shareholder can front-run the call to the `remove_shareholder` function by submitting a request for a partial/full liquidation, cash purchase, or share transfer. It is important to note that if self-service is disabled, this attack vector is harder to trigger. Additionally, the malicious shareholder must perform the front-running attack once a day to ensure that there is always a pending transaction.

Finally, the `only_when_shareholder_has_no_shares` check can be triggered by sending dust to the address (`1e-share_token_decimals`), which can also prevent the removal of the shareholder.

Exploit Scenario

Eve, a malicious shareholder, is about to be removed as a shareholder from the system. She notices a call to `remove_shareholder` in the public mempool. Eve front-runs this

transaction with a call to `request_self_service_cash_purchase` with which she purchases 1 FA unit. This prevents Eve from being removed as a shareholder.

Recommendations

Short term, remove the call to `only_when_shareholder_has_no_pending_transactions` in the `remove_shareholder` function. Additionally, delete all pending transactions when a shareholder is being removed.

Long term, always analyze contract functions that could be affected by front-running attacks. Additionally, document any front-running-related risks.

5. Insufficient event generation

Severity: Informational

Difficulty: Low

Type: Auditing and Logging

Finding ID: TOB-FT-APTOS-5

Target: funds/sources/fund_token.move,
multisig/sources/access_control.move,
funds/sources/erc20_wrapper.move

Description

Multiple critical operations do not emit events. As a result, it will be difficult to review the modules' behavior for correctness once they have been deployed.

Events generated during transaction execution aid in monitoring, baselining of behavior, and detection of suspicious activity. Without events, users and blockchain-monitoring systems cannot easily detect behavior that falls outside the baseline conditions; malfunctioning modules and attacks could go undetected.

The following operations should trigger events:

- fund_token::initialize
- fund_token::update_last_price
- fund_token::enable_self_service
- fund_token::disable_self_service
- access_control::generate_shared_account_signer
- access_control::init_access_control_internal
- erc20_wrapper.initialize

Recommendations

Short term, add events for all operations that could contribute to a higher level of monitoring and alerting.

Long term, consider using a blockchain-monitoring system to track any suspicious behavior in the contracts. The system relies on several modules to behave as expected. A monitoring mechanism for critical events would quickly detect any compromised system components.

6. Frozen shareholders can be distributed dividends

Severity: Informational

Difficulty: Low

Type: Data Validation

Finding ID: TOB-FT-APTOS-6

Target: fund/sources/fund_token.move, fund/sources/erc20_wrapper.move

Description

Dividend distribution does not exclude frozen shareholders; this intuitively contradicts the idea that frozen shareholders should not be able to interact with the system.

Whenever the admin executes a dividend distribution through the `distribute_dividends` or through the `end_of_day` functions, they have to input a list of accounts (shareholders addresses) that will be the recipients of said distribution.

Both of the aforementioned functions rely on the internal function `process_dividends`, which is highlighted in the figure below.

```
1093 fun process_dividends(  
1094     account: address,  
1095     rate: u64,  
1096     price: u64,  
1097     memo: String,  
1098 ) acquires ManagedShareHolders, FundManagement {  
1099     only_when_account_exists(account);  
1100  
1101     let fund = borrow_global<FundManagement>(@ft_funds);  
1102     let symbol: vector<u8> = fund.symbol;  
1103  
1104     if (erc20_wrapper::balance(account, symbol) > 0) {  
1105         let resource_signer =  
1106         &account::create_signer_with_capability(&fund.resource_signer_cap);  
1107         // calculate dividends = (account's balance * rate) / price  
1108         let shares = (erc20_wrapper::balance(account, symbol) * rate) /  
1109         price;  
1110         // mint shares  
1111         erc20_wrapper::mint(resource_signer, account, shares, symbol);  
1112         [...]  
1125     }
```

Figure 6.1: `process_dividends` function in `fund_token.move`#L1093–L1125

As shown in the figure above, the function will first check that the shareholder account exists and that the account holds shares, and then will call the mint function of the `erc20_wrapper` module.

```
103     public fun mint(  
104         admin: &signer,  
105         to: address,  
106         amount: u64,  
107         token_symbol: vector<u8>,  
108     ) acquires ManagedFungibleAsset {  
109         let asset = get_metadata(token_symbol);  
110         let managed_fungible_asset = authorized_borrow_refs(admin, asset);  
111         let to_wallet = primary_fungible_store::ensure_primary_store_exists(to,  
asset);  
112         let fa = fungible_asset::mint(&managed_fungible_asset.mint_ref, amount);  
113         fungible_asset::deposit_with_ref(&managed_fungible_asset.transfer_ref,  
to_wallet, fa);  
114     }
```

Figure 6.2: mint function in `erc20_wrapper.move#L103-L114`

As shown in the above figure, the mint function will not perform any checks on the to address, and it will use the fungible_asset module `deposit_with_ref` function. As shown in the figure below, this function will ignore whether the recipient is frozen.

```
926     /// Deposit the fungible asset into the `store` ignoring `frozen`.  
927     public fun deposit_with_ref<T: key>(  
928         ref: &TransferRef,  
929         store: Object<T>,  
930         fa: FungibleAsset  
931     ) acquires FungibleStore, ConcurrentFungibleBalance {
```

Figure 6.3: deposit_with_ref function in `fungible_asset.move#L926-L931`

All in all, this means that a frozen shareholder can be the target of a dividend distribution.

Note that the client has stated that this behavior is expected; however, we still wish to highlight it, as it can lead to confusion.

Recommendations

Short term, consider whether frozen shareholders should be able to receive dividends, and implement the necessary checks if not.

Long term, thoroughly document the intended behavior so that users are aware of it.

7. Insufficient checks during request settlement

Severity: High

Difficulty: Low

Type: Data Validation

Finding ID: TOB-FT-APTOS-7

Target: funds/sources/fund_token.move

Description

Currently, whenever a request is created, a number of checks are performed before the request is added to the queue. However, the checks are not performed again at the moment the request is processed, which can lead to unexpected behavior.

Whenever a request is created, either by the admin or by the user through the self-service, a number of checks are performed based on the type of request that is being created.

For example, when a shareholder requests to transfer their shares to another shareholder, various checks—such as whether the sender has enough balance, or that neither of the shareholders is frozen—are performed (figure 7.1).

```
532     public entry fun request_share_transfer(  
533         caller: &signer,  
534         account: address,  
535         destination: address,  
536         shares: u64,  
537         memo: String,  
538     ) acquires ManagedShareHolders, FundManagement {  
539         only_whitelisted(caller);  
540         only_when_initialized();  
541         only_when_account_exists(account);  
542         only_when_account_exists(destination);  
543         only_when_account_not_frozen(account);  
544         only_when_account_not_frozen(destination);  
545         only_when_shareholder_has_shares(account);  
546         only_higher_than_zero(shares);  
547         only_when_shareholder_has_enough_shares(account, shares);  
548     }  
549     create_pending_transaction(account, destination, TX_TYPE_SHARE_TRANSFER,  
550     shares, false, memo);  
551 }
```

Figure 7.1: The request_share_transfer function fund_token.move#L532–L550

Another example is whenever a user requests a cash liquidation. In this case, the user states an amount of cash they would like to receive, and the fund will burn user shares equivalent to that cash amount.

For this example, the function will check that the user is not frozen and that it has enough shares to burn for the amount of cash they would like to receive using the current price (figure 7.2).

```
490 public entry fun request_cash_liquidation(  
491     caller: &signer,  
492     account: address,  
493     amount: u64,  
494     memo: String,  
495 ) acquires ManagedShareHolders, FundManagement {  
496     only_whitelisted(caller);  
497     only_when_initialized();  
498     only_when_account_exists(account);  
499     only_when_account_not_frozen(account);  
500  
501     let managed = borrow_global_mut<ManagedShareHolders>(@ft_funds);  
502     let price = managed.last_price;  
503  
504     only_valid_price(price);  
505     only_when_shareholder_has_enough_amount(account, amount, price);  
506  
507     create_pending_transaction(account, @0x0, TX_TYPE_CASH_LIQUIDATION,  
amount, false, memo);  
508 }
```

Figure 7.2: request_cash_liquidation function in fund_token.move#L490–L508

The aforementioned checks prevent unwanted behavior, such as frozen accounts interacting with the fund or users creating requests that cannot be fulfilled. However, as shown in the figure below, these checks are not present during the settlement of requests.

```
1127 fun process_settlements(  
1128     account: address,  
1129     date: u64,  
1130     price: u64,  
1131     memo: String,  
1132 ) acquires ManagedShareHolders, FundManagement {  
1133     only_when_account_exists(account);  
1134  
1135     let fund = borrow_global<FundManagement>(@ft_funds);  
1136     let managed = borrow_global_mut<ManagedShareHolders>(@ft_funds);  
1137     let shareholders = &mut managed.shareholders;  
1138     let shareholder = smart_table::borrow_mut(shareholders, account);  
1139     let resource_signer =  
&account::create_signer_with_capability(&fund.resource_signer_cap);  
1140 }
```



```

1141     only_max_size_batch(vector::length(&shareholder.request_ids),
MAX_BATCH_SIZE);
1142
1143     while (!vector::is_empty(&shareholder.request_ids)) {
1144         let request_id = vector::pop_back<u64>(&mut
shareholder.request_ids);
1145
1146         // check if transaction exists
1147         if (smart_table::contains(&managed.pending_transactions,
request_id)) {
1148             // get transaction
1149             let tx = smart_table::borrow(&managed.pending_transactions,
request_id);
1150
1151             only_supported_transaction_types(tx.tx_type);
1152
1153             // the requested transaction has a prior date ?
1154             if (tx.date <= date) {
1155                 let shares = 0;
1156                 if (tx.tx_type == TX_TYPE_CASH_LIQUIDATION) {
1157                     if (tx.amount > 0) {
1158                         shares = calculate_shares(tx.amount, price,
erc20_wrapper::decimals(fund.symbol));
1159                         erc20_wrapper::burn(resource_signer, account,
shares, fund.symbol);
1160                     }
1161                 }
1162                 [...]
1175                 else if (tx.tx_type == TX_TYPE_SHARE_TRANSFER) {
1176                     erc20_wrapper::transfer(
1177                         resource_signer,
1178                         tx.account,
1179                         tx.destination,
1180                         tx.amount,
1181                         fund.symbol
1182                     );
1183                 }
1184                 [...]

```

Figure 7.3: *process_settlements* function *fund_token.move*#L1127–L1244

This behavior is problematic because the state of the fund can change between the moment a request is created and the moment it is fulfilled by the admin. For example, an admin might freeze a shareholder's account while they have pending requests, thus breaking the assumption that frozen shareholders cannot interact with the system.

This is also problematic because it can lead to the creation of requests that cannot be fulfilled because requests validate the current state of the fund, not the state of the fund after previous requests have been fulfilled. For example, a user could technically create multiple requests to transfer their entire balance to different shareholders; however, the moment the first request is fulfilled by the *process_settlements* function, all

subsequent requests will fail, leading to a temporary DoS until the invalid requests are canceled.

Finally, this can also be applied to the share price, which is updated before the requests are settled; this could lead to undefined behavior. For example, a user may request a share purchase but will receive more or fewer shares than they expected because the price fluctuated when the request was fulfilled.

Exploit Scenario

Eve, a malicious user, creates an account and interacts with the fund by making a number of cash purchases. After some time, the AML policy is triggered, and Eve's account has to be frozen.

Before her account is frozen, Eve creates a series of self-service requests that distribute her share balance across many other accounts. The admin freezes Eve's account, effectively preventing her from interacting with the fund anymore. However, unbeknownst to the administrators, during the execution of the `end_of_day` function, Eve's pending requests are executed, effectively distributing her funds across many accounts and potentially giving her a chance to cash out.

Recommendations

Short term, perform additional checks for each request type during the settlement of requests to avoid unwanted behavior and prevent failing requests from triggering a DoS (e.g., by catching the error).

Note that choosing the right policy (i.e., catching the error and settling the next request from the same shareholder instead of skipping the shareholder altogether) is not a decision that should be taken lightly, and whatever decision is made should be thoroughly documented.

Long term, thoroughly document this behavior and incorporate additional unit tests to verify that the intended behavior is preserved during each one of the request settlements.

8. A high number of pending requests could lead to a temporary denial of service

Severity: Informational

Difficulty: High

Type: Denial of Service

Finding ID: TOB-FT-APTOS-8

Target: funds/sources/fund_token.move,

Description

Because the creation of requests and their settlement are separated in two different steps, the creation of requests does not change the fund's state. As a result, a user can create requests that are not fulfillable when they are settled; additionally, in practice, the number of requests they can create is limited only by the pending request vector size.

```
390     public entry fun settle_transactions(  
391         caller: &signer,  
392         date: u64,  
393         price: u64,  
394         accounts: vector<address>,  
395         memo: String,  
396     ) acquires ManagedShareHolders, FundManagement {  
397         only_whitelisted(caller);  
398         only_when_initialized();  
399         only_valid_price(price);  
400         only_max_size_batch(vector::length(&accounts), MAX_BATCH_SIZE);  
401  
402         update_last_price(caller, price);  
403  
404         vector::for_each(accounts, |account| {  
405             process_settlements(  
406                 account,  
407                 date,  
408                 price,  
409                 memo,  
410             )  
411         });  
412     }
```

Figure 8.1: `settle_transactions` function in `fund_token.move`#L390–L412

This means that a malicious user (or a faulty API) could create a high number of pending requests that at the moment of fulfillment will make the transaction run out of gas.

Note that, if exploited, this issue would lead to a temporary denial of service that would last only until the pending requests are canceled.

Exploit Scenario

Eve, a malicious user, creates a very high number of requests that, when settled, lead to an out-of-gas error.

Recommendations

Short term, consider adding an upper bound to the number of requests a user may create in a day. Because the fund is expected to clear all pending requests daily and to distribute dividends daily, this solution should not interfere with the user experience, as the limit would be reset daily.

9. Pending requests are not settled in the order in which they were created

Severity: High

Difficulty: Low

Type: Data Validation

Finding ID: TOB-FT-APTOS-9

Target: funds/sources/fund_token.move

Description

Users can interact with the fund by creating different requests (e.g., requesting a share purchase or a liquidation). However, during the settlement of the requests, they are processed backward (i.e., from newest to oldest) instead of being executed in the order the user created them. This issue can lead to unexpected behavior.

```
1127 fun process_settlements(  
1128     account: address,  
1129     date: u64,  
1130     price: u64,  
1131     memo: String,  
1132 ) acquires ManagedShareHolders, FundManagement {  
1133     only_when_account_exists(account);  
1134  
1135     let fund = borrow_global<FundManagement>(@ft_funds);  
1136     let managed = borrow_global_mut<ManagedShareHolders>(@ft_funds);  
1137     let shareholders = &mut managed.shareholders;  
1138     let shareholder = smart_table::borrow_mut(shareholders, account);  
1139     let resource_signer =  
&account::create_signer_with_capability(&fund.resource_signer_cap);  
1140  
1141     only_max_size_batch(vector::length(&shareholder.request_ids),  
MAX_BATCH_SIZE);  
1142  
1143     while (!vector::is_empty(&shareholder.request_ids)) {  
1144         let request_id = vector::pop_back<u64>(&mut  
shareholder.request_ids);
```

Figure 9.1: `process_settlements` function in `fund_token.move`#L1127–L1144

As shown in the figure above, requests are removed from the vector of pending requests using the `pop_back` function, which removes and returns the last element of the vector.

Exploit Scenario

Alice, a user of the fund who currently owns five shares, creates a number of requests during the day:

1. She makes a cash purchase of a number of 10 shares.
2. She transfers some shares to another shareholder.
3. She requests a partial liquidation of 11 shares.

When Alice's requests are processed at the end of the day, the first request that will be fulfilled will be the partial liquidation, which will fail because Alice has only five shares at the moment of the fulfillment.

Recommendations

Short term, process requests in the order in which they were created.

Long term, implement additional unit tests to check that the pending requests are processed in the order in which they were created.

10. The order in which the admin inputs the accounts to the settlement function can alter the outcome of the settlement

Severity: Informational

Difficulty: Low

Type: Undefined Behavior

Finding ID: TOB-FT-APTOS-10

Target: funds/sources/fund_token.move

Description

The state of the fund after each daily settlement can vary depending on the order in which accounts are settled by the administrator.

Whenever an administrator calls the `end_of_day` or `settle_transactions` function, they input a vector of accounts to it.

```
390 public entry fun settle_transactions(  
391     caller: &signer,  
392     date: u64,  
393     price: u64,  
394     accounts: vector<address>,  
395     memo: String,  
396 ) acquires ManagedShareHolders, FundManagement {  
397     only_whitelisted(caller);  
398     only_when_initialized();  
399     only_valid_price(price);  
400     only_max_size_batch(vector::length(&accounts), MAX_BATCH_SIZE);  
401  
402     update_last_price(caller, price);  
403  
404     vector::for_each(accounts, |account| {  
405         process_settlements(  
406             account,  
407             date,  
408             price,  
409             memo,  
410         )  
411     });  
412 }
```

Figure 10.1: `settle_transactions` function in `fund_token.move`#L390–L412

As described below, the vector's order will determine the order in which accounts are settled through the `process_settlements` function; however, just as when transactions are processed on a blockchain, the order will determine the outcome.

For example, consider the following scenario:

Alice and Bob are users of the fund, each holding 10 shares.

1. Alice creates a request to transfer 1 share to Bob.
2. Bob creates a request to liquidate 11 shares.

During the day settlement, if Alice's account is settled first, Bob's request will succeed; otherwise, Bob will have to wait another day to liquidate his shares.

Note that this is not a security issue, but a subtlety of the two-step request settlement process that may lead to a bad user experience.

Recommendations

Short term, a possible solution could be implementing `request_ids` as nonces (i.e., monotonically increasing), which are increased on every new request. Additionally, instead of settling all pending requests for all users, a number of requests could be settled every day (e.g., every 10 or 100 days) starting from the last `request_id` that was processed. This will ensure that transactions are settled on a first come, first served basis (based on their nonce), which would mitigate this issue.

Note that a solution like this one should also consider imposing daily limits on users to prevent them from filling all of the "request space."

Long term, document this behavior so that users are aware of it.

11. Creating requests just before the end of day allows shareholders to instantly settle them

Severity: Informational

Difficulty: Low

Type: Undefined Behavior

Finding ID: TOB-FT-APTOS-11

Target: funds/sources/fund_token.move

Description

Once a day, the fund administrator calls the `end_of_day` function. This function distributes dividends to all current shareholders of the fund and then settles their pending requests.

```
415     public entry fun end_of_day(  
416         caller: &signer,  
417         rate: u64,  
418         price: u64,  
419         accounts: vector<address>,  
420         memo: String,  
421     ) acquires ManagedShareHolders, FundManagement {  
422         only_whitelisted(caller);  
423         only_when_initialized();  
424         only_valid_rate(rate);  
425         only_valid_price(price);  
426         only_max_size_batch(vector::length(&accounts), MAX_BATCH_SIZE);  
427  
428         update_last_price(caller, price);  
429  
430         vector::for_each(accounts, |account| {  
431             process_dividends(  
432                 account,  
433                 rate,  
434                 price,  
435                 memo,  
436             );  
437             process_settlements(  
438                 account,  
439                 rate,  
440                 price,  
441                 memo,  
442             );  
443             let params = format2(&b"rate = {} price = {}", rate, price);  
444             add_activity(account, timestamp::now_seconds(), 0,  
TX_TYPE_END_OF_DAY, utf8(b""), params);  
445         });  
446     }
```

Figure 11.1: `end_of_day` function in `fund_token.move#L415–L446`

As shown in the figure above, the admin inputs a number of accounts that will have their transactions settled through the `process_settlements` function, and also inputs a date. (Note that there is a bug in the current implementation, but we will ignore it for now). All pending requests, from each one of the accounts, will be settled if they were created at or before the date input by the admin.

```
1127 fun process_settlements(  
1128     account: address,  
1129     date: u64,  
1130     price: u64,  
1131     memo: String,  
1132 ) acquires ManagedShareHolders, FundManagement {  
1133     only_when_account_exists(account);  
1134  
1135     let fund = borrow_global<FundManagement>(@ft_funds);  
1136     let managed = borrow_global_mut<ManagedShareHolders>(@ft_funds);  
1137     let shareholders = &mut managed.shareholders;  
1138     let shareholder = smart_table::borrow_mut(shareholders, account);  
1139     let resource_signer =  
&account::create_signer_with_capability(&fund.resource_signer_cap);  
1140  
1141     only_max_size_batch(vector::length(&shareholder.request_ids),  
MAX_BATCH_SIZE);  
1142  
1143     while (!vector::is_empty(&shareholder.request_ids)) {  
1144         let request_id = vector::pop_back<u64>(&mut  
shareholder.request_ids);  
1145  
1146         // check if transaction exists  
1147         if (smart_table::contains(&managed.pending_transactions,  
request_id)) {  
1148             // get transaction  
1149             let tx = smart_table::borrow(&managed.pending_transactions,  
request_id);  
1150  
1151             only_supported_transaction_types(tx.tx_type);  
1152  
1153             // the requested transaction has a prior date ?  
1154             if (tx.date <= date) {  
[...]
```

Figure 11.2: `process_settlement` function in `fund_token.move#L1127–L1154`

However, because there is supposed to be a delay before creation and settlement, this behavior raises a potential concern: if a user front-runs the `end_of_day` function with all of their requests, the user would have all their requests settled near-instantly.

This behavior, in theory, should not be a problem; however, if the two-step request settlement has an operational reason to exist (e.g., because some event must happen off-chain before a request can be processed), then other problems may exist that stem from this “near-instant” request settlement that we are unable to discern within the current context.

Note that, assuming that the `end_of_day` function is akin to a cron job (i.e., a scheduled process), it should be trivial to “exploit” this behavior.

Recommendations

Short term, document this behavior and whether it can lead to potential harm for the system.

12. The end_of_day function is broken

Severity: High

Difficulty: Low

Type: Data Validation

Finding ID: TOB-FT-APTOS-12

Target: funds/sources/fund_token.move

Description

Once a day, the fund administrator calls the end_of_day function. This function distributes dividends to all current shareholders of the fund and then settles their pending requests.

```
415 public entry fun end_of_day(  
416     caller: &signer,  
417     rate: u64,  
418     price: u64,  
419     accounts: vector<address>,  
420     memo: String,  
421 ) acquires ManagedShareHolders, FundManagement {  
422     only_whitelisted(caller);  
423     only_when_initialized();  
424     only_valid_rate(rate);  
425     only_valid_price(price);  
426     only_max_size_batch(vector::length(&accounts), MAX_BATCH_SIZE);  
427  
428     update_last_price(caller, price);  
429  
430     vector::for_each(accounts, |account| {  
431         process_dividends(  
432             account,  
433             rate,  
434             price,  
435             memo,  
436         );  
437         process_settlements(  
438             account,  
439             rate,  
440             price,  
441             memo,  
442         );  
443         let params = format2(&b"rate = {} price = {}", rate, price);  
444         add_activity(account, timestamp::now_seconds(), 0,  
TX_TYPE_END_OF_DAY, utf8(b""), params);  
445     });  
446 }
```

Figure 12.1: end_of_day function in fund_token.move#L415–L446

To this function, the admin inputs an array of accounts, the new share price, the rate to calculate the new shares to be minted, and a memo.

However, the process_settlements function, which is called internally, takes an account for which to settle the pending requests, the new share price, the memo, and a date that is the timestamp that determines whether or not a pending transaction gets settled.

```
1127 fun process_settlements(  
1128     account: address,  
1129     date: u64,  
1130     price: u64,  
1131     memo: String,  
1132 ) acquires ManagedShareHolders, FundManagement {
```

Figure 12.2: process_settlement function in fund_token.move#L1127–L1132

However, as shown in the figures above, the end_of_day function uses the rate as the value for the date parameter in the process_settlements transaction, which is wrong and will lead to no transactions being settled.

Recommendations

Short term, include a date parameter in the function.

Long term, include additional unit tests to validate the expected behavior of the system; a very simple unit test would have caught this bug.

13. Multisig integrity checks are not performed on addresses with PERMISSION_LEVEL_ALL role

Severity: Medium

Difficulty: Low

Type: Data Validation

Finding ID: TOB-FT-APTOS-13

Target: funds/sources/fund_token.move

Description

One of the most recent features added to the access control module allows the module's administrator to "check the integrity" of multisig accounts added to the system.

To achieve that, a new role PERMISSION_LEVEL_WHITELISTED_OWNER was added, which represents a whitelisted multisig owner.

```
59    /// It allows the validation of the multisig_account owners for enhanced
    protection
60    /// This role should be only granted to owners of any multisig_account
61    /// with role PERMISSION_LEVEL_FUND_MANAGER
62    const PERMISSION_LEVEL_WHITELISTED_OWNER: u8 = 2;
```

Figure 13.1: PERMISSION_LEVEL_WHITELISTED_OWNER role in access_control.move#L59-L62

Additionally, the number of signatures required for a multisig to operate is stored. These two additions bring another layer of security to the system, as arbitrary changes to the whitelisted multisigs made by their owners (such as adding or removing signers or altering the threshold) without the administrator's approval will make them unable to interact with the system.

```
104    struct AccessControl has key {
105        /// Role based Access Control List for signer verification
106        /// Note: This has some extra storage gas upfront to support fast lookups
    and large lists
107        permission_list: SmartTable<address, u8>,
108        /// Additional list that could be used to store a multisig_account
    threshold or another
109        /// value related with the given (key) address
110        permission_metadata: SmartTable<address, u64>,
111        /// Extend ref allows us to do anything as the object
112        extend_ref: ExtendRef,
113        /// This is made to gate transfers so they can't be made without this
    contract
```

```

114     transfer_ref: TransferRef,
115 }

```

Figure 13.2: AccessControl struct in access_control.move#L104–L115

The fund module will perform the integrity check whenever a whitelisted multisig interacts with it; however, as shown in the figure below, the `check_multisig_integrity` function is only called when the calling multisig has a `PERMISSION_LEVEL_FUND_MANAGER` role and not when the role is `PERMISSION_LEVEL_ALL`.

```

1511 inline fun check_authorized_policy(obj: Object<AccessControl>, caller:
1512 &signer) {
1513     // we assume a multisig_account is the caller so it will revert if
1514     // that's not the case
1515     if(is_permission_level_fund_manager(obj, signer::address_of(caller))) {
1516         check_multisig_integrity(obj, caller);
1517     }
1518 }
1519 // Revert if the integrity of the caller multisig_account cannot be
1520 // ensured,
1521 // integrity is checked against the permission and metadata lists in the
1522 // access_control module.
1523 inline fun check_multisig_integrity(obj: Object<AccessControl>, caller:
1524 &signer) {
1525     let multisig_address = signer::address_of(caller);
1526     // verify threshold integrity
1527     let threshold =
1528 multisig_account::num_signatures_required(multisig_address);
1529     assert!(get_metadata_value(obj, multisig_address) == threshold,
1530 EINTEGRITY_VIOLATION);
1531     // verify owners integrity
1532     vector::for_each(multisig_account::owners(multisig_address), |ms_owner|
1533 {
1534     1527         assert!(is_permission_level_whitelisted_owner(obj, ms_owner),
1535 EINTEGRITY_VIOLATION);
1536     1528     });
1537     1529 }

```

Figure 13.3: Integrity checks performed in fund_token.move#L1511–L1529

This is problematic, because there is no mention in the available documentation that a multisig cannot be given the `PERMISSION_LEVEL_ALL` role (something that is highlighted for other roles) and therefore skip the new integrity checks.

Exploit Scenario

The administrator grants the `PERMISSION_LEVEL_ALL` role to a 3-of-3 multisig controlled by Alice, Bob, and Eve.

Alice and Eve collude to kick Bob out of the multisig; however, this does not prevent their multisig from interacting with the system as there are no integrity checks.

Recommendations

Short term, include the integrity check for multisigs with the PERMISSION_LEVEL_ALL role.

Long term, thoroughly document whether the PERMISSION_LEVEL_ALL role can be assigned to multisigs. If it cannot, then there is no need for an integrity check; otherwise, include the integrity check.

Note that if the PERMISSION_LEVEL_ALL role can be added to non-multisigs, the integrity check should not be performed on them. A possible way in which this integrity check could discern between multisigs and non-multisigs would be by using a threshold of 0 in the metadata field to indicate that the account is not a multisig.

14. The `recover_account` and `recover_assets` functions do not check whether the source and destination addresses are the same

Severity: Low

Difficulty: Low

Type: Data Validation

Finding ID: TOB-FT-APTOS-14

Target: `funds/sources/fund_token.move`

Description

The administrator can use the `recover_asset` and `recover_account` functions to transfer shares from a source account to a destination account. The main difference between them is that `recover_account` transfers all of the available shares of the source account and then removes the shareholder from the system.

```
400 public entry fun recover_account(  
401     caller: &signer,  
402     source: address,  
403     destination: address,  
404     memo: String,  
405 ) acquires ManagedShareHolders, FundManagement {  
406     only_whitelisted(caller);  
407     only_when_initialized();  
408     only_when_account_exists(source);  
409     only_when_account_exists(destination);  
410     only_when_shareholder_has_shares(source);  
[...]  
439 }
```

Figure 14.1: `recover_account` function in `fund_token.move`#L400–L439

```
445 public entry fun recover_asset(  
446     caller: &signer,  
447     source: address,  
448     destination: address,  
449     shares: u64,  
450     memo: String,  
451 ) acquires ManagedShareHolders, FundManagement {  
452     only_whitelisted(caller);  
453     only_when_initialized();  
454     only_when_account_exists(source);  
455     only_when_account_exists(destination);  
456     only_when_shareholder_has_enough_shares(source, shares);  
[...]  
476 }
```

Figure 14.2: recover_asset function in fund_token.move#L445-L476

However, as shown in the figures above, there is no check preventing source and destination from being the same address; this essentially results in no-op, as the balance would remain the same. Additionally, in the case of recover_account, the function will result in the removal of the source address from the shareholder list even though it still has a share balance.

Note that the issue is not of higher severity because the share balance is not lost/burned and a call to add_shareholder would restore the account back to its previous state.

Recommendations

Short term, include an additional check to prevent the source and destination addresses from being the same.

Long term, include additional unit tests to validate the expected behavior of the system.

15. Account recovery can be prevented by creating a pending transaction

Severity: Informational

Difficulty: Low

Type: Data Validation

Finding ID: TOB-FT-APTOS-15

Target: funds/sources/fund_token.move

Description

The `recover_account` function is used to transfer all of the shares from a source account to a destination account and then remove the source account from the shareholder list.

One of the checks made by the function prevents its execution if the source account has pending transactions. If self-service is enabled, this could allow a malicious user to prevent the administrator from executing the function by simply creating new requests.

```
400     public entry fun recover_account(  
401         caller: &signer,  
402         source: address,  
403         destination: address,  
404         memo: String,  
405     ) acquires ManagedShareHolders, FundManagement {  
406         only_whitelisted(caller);  
407         only_when_initialized();  
408         only_when_account_exists(source);  
409         only_when_account_exists(destination);  
410         only_when_shareholder_has_shares(source);  
411  
412         let fund_management_address = fund_management_address();  
413         let managed =  
borrow_global_mut<ManagedShareHolders>(fund_management_address);  
414         let shareholder = smart_table::borrow_mut(&mut managed.shareholders,  
source);  
415  
416         only_when_shareholder_has_no_pending_transactions(shareholder);
```

Figure 15.1: `recover_account` function in `fund_token.move`#L400–L416

Note that an administrator could always call `recover_assets` instead to recover the shares, then remove all pending transactions from the account, and then remove the shareholder through `remove_shareholder`; however, since the function seems to be intended to do all of that atomically, we want to highlight this behavior.

Recommendations

Short term, iterate over shareholder pending transactions (if any), removing them from the list, and then transfer their shares to the destination address; finally, remove the shareholder from the system.

Long term, thoroughly document the intended behavior of the function.

16. Beware of changing token decimals

Severity: Informational

Difficulty: Low

Type: Undefined Behavior

Finding ID: TOB-FT-APTOS-16

Target: funds/sources/erc20_wrapper.move

Description

The `erc20_wrapper` module now includes a function to change the metadata associated with a given token.

```
141    /// Allows the mutation of the fungible asset metadata except the token
symbol.
142    // Because primary stores have deterministic addresses it could be
problematic
143    // to change the symbol that was originally used to create the object.
144    public fun mutate_metadata(
145        admin: &signer,
146        current_symbol: vector<u8>,
147        new_name: Option<String>,
148        new_decimals: Option<u8>,
149        new_icon_uri: Option<String>,
150        new_project_uri: Option<String>,
151    ) acquires ManagedFungibleAsset {
152        let asset = get_metadata(current_symbol);
153        let managed_fungible_asset = authorized_borrow_refs(admin, asset);
154        fungible_asset::mutate_metadata(
155            &managed_fungible_asset.mutate_ref,
156            new_name,
157            option::none(),
158            new_decimals,
159            new_icon_uri,
160            new_project_uri
161        );
162    }
```

Figure 16.1: `mutate_metadata` function in `erc20_wrapper.move` #L141–L162

However, as shown in the figure above, the metadata update allows the administrator to change the token decimals. This operation is risky, as it could impact the accounting of off-chain services or even user transactions.

Recommendations

Short term, consider whether allowing the change of the token decimals is necessary. If it is, then thoroughly document the associated risks and how to mitigate them (e.g., by informing users or creating a process to update the metadata).

17. The addition of negative yield, asset recovery, and share adjustment can also trigger TOB-FT-APTOS-7

Severity: Informational

Difficulty: Low

Type: Undefined Behavior

Finding ID: TOB-FT-APTOS-17

Target: funds/sources/fund_token.move

Description

As highlighted in [TOB-FT-APTOS-7](#), a number of operations can influence the settlement of pending transactions (even pending transactions themselves), as these transactions do not alter the state of the fund until they are settled by the administrator.

The addition of the negative yield feature in `process_dividends` and the ability to change a given shareholder's balance either through `recover_assets` or `adjust_shares` introduce new ways in which the outcome of pending transactions can be affected before they are settled and can lead to unexpected behavior.

Recommendations

Short term, thoroughly document this behavior so that users and administrators are aware of its risks.

When performing administrative operations, beware of how these can affect pending transactions.

18. There is no association between multisig addresses and their underlying configuration

Severity: Medium

Difficulty: High

Type: Undefined Behavior

Finding ID: TOB-FT-APTOS-18

Target: funds/sources/access_control.move

Description

One of the most recent features added to the access control module allows the module's administrator to "check the integrity" of multisig accounts added to the system.

To achieve that, a new role `PERMISSION_LEVEL_WHITELISTED_OWNER` was added, which represents a whitelisted multisig owner.

```
55    /// It allows the holder of the role to generate a common signer for this
module and also
56    /// to interact with the fund_token module similar to
PERMISSION_LEVEL_FUND_MANAGER
57    /// This role should be treated carefully
58    const PERMISSION_LEVEL_ALL: u8 = 1;
59    /// It allows the validation of the multisig_account owners for enhanced
protection
60    /// This role should be only granted to owners of any multisig_account
61    /// with role PERMISSION_LEVEL_FUND_MANAGER
62    const PERMISSION_LEVEL_WHITELISTED_OWNER: u8 = 2;
63    /// It allows the holder of the role to interact with the fund_token module
64    /// This role should be only granted to a multisig_account
65    const PERMISSION_LEVEL_FUND_MANAGER: u8 = 3;
```

Figure 18.1: `PERMISSION_LEVEL_WHITELISTED_OWNER` role in `access_control.move`#L55-L65

Additionally, the number of signatures required for a multisig to operate is stored. These two additions bring another layer of security to the system, as arbitrary changes to the whitelisted multisigs made by their owners (such as adding or removing signers or altering the threshold) without the administrator's approval will make them unable to interact with the system.

```
104    struct AccessControl has key {
105        /// Role based Access Control List for signer verification
106        /// Note: This has some extra storage gas upfront to support fast lookups
and large lists
```



```

107     permission_list: SmartTable<address, u8>,
108     /// Additional list that could be used to store a multisig_account
threshold or another
109     /// value related with the given (key) address
110     permission_metadata: SmartTable<address, u64>,
111     /// Extend ref allows us to do anything as the object
112     extend_ref: ExtendRef,
113     /// This is made to gate transfers so they can't be made without this
contract
114     transfer_ref: TransferRef,
115 }

```

Figure 18.2: AccessControl struct in access_control.move#L104–L115

When a new account is added to the system, the `permission_list` is populated with its associated role (e.g., owner, fund manager.), and its associated metadata (if any) will be added to the `permission_metadata` list.

```

157     /// Add an account to the access control list
158     /// The account will be granted the given `role` associated with the
permission level defined in this module
159     ///
160     /// @param sender the calling account, it should be the admin of this module
161     /// @param shared_account the address of the internal object holding access
control list
162     /// @param account the address of the account to add to the access control
list of this module
163     /// @param role the role corresponding to the defined permission level
164     public entry fun add_account(
165         sender: &signer,
166         shared_account: Object<AccessControl>,
167         account: address,
168         role: u8,
169         metadata: u64,
170     ) acquires AccessControl {
171         assert!(!is_on_allowlist(shared_account, account),
EACCOUNT_ALREADY_EXISTS);
172         let management = assert_is_admin(sender, shared_account);
173         smart_table::add(&mut management.permission_list, account, role);
174         smart_table::add(&mut management.permission_metadata, account, metadata);
175         aptos_framework::event::emit(AddSignerEvent {
176             account,
177             role
178         });
179     }

```

Figure 18.3: add_account function role in access_control.move#L157–L179

However, in practice, there is a fundamental flaw with how this feature is implemented: a given multisig account is not associated with its configuration. Instead, all accounts—whether or not they are multisigs—are stored in the `permission_list`.

Therefore, if an account is whitelisted as an owner, it will also be whitelisted for all multisigs in the list, bypassing the intended security layer.

To illustrate the issue, consider the following hypothetical example:

Three multisigs—A, B, and C —have been added to the system by the administrator with a `PERMISSION_LEVEL_FUND_MANAGER` role and a threshold of 2. Multisig A has the following owners [Alice, Bob], multisig B has [Charlie, Eve], and multisig C has [Mallory, Fred]. All of these signers have been added to the permission list with the `PERMISSION_LEVEL_WHITELISTED_OWNER` role.

Some time passes, and Charlie has his owner roles revoked, essentially preventing multisig B from interacting with the system, as they would fail the integrity check. Additionally, Charlie and Eve sign the removal of Charlie's address from the multisig, as Eve is no longer an owner. Finally, Eve and Alice collude and perform the following action:

- Eve adds Alice to multisig B.

Now, whenever the integrity check is performed (as illustrated in the figure below), it will pass successfully because, even though Alice was not whitelisted as owner of multisig B, there is no association between multisigs and their owners. Instead, the only check that owners need to pass is that they are whitelisted as owners in the general sense (i.e., that they have the `PERMISSION_LEVEL_WHITELISTED_OWNER` role).

```
1520 inline fun check_multisig_integrity(obj: Object<AccessControl>, caller:
&signer) {
1521     let multisig_address = signer::address_of(caller);
1522     // verify threshold integrity
1523     let threshold =
multisig_account::num_signatures_required(multisig_address);
1524     assert!(get_metadata_value(obj, multisig_address) == threshold,
EINTEGRITY_VIOLATION);
1525     // verify owners integrity
1526     vector::for_each(multisig_account::owners(multisig_address), |ms_owner|
{
1527         assert!(is_permission_level_whitelisted_owner(obj, ms_owner),
EINTEGRITY_VIOLATION);
1528     });
1529 }
```

Figure 18.4: `check_multisig_integrity` function in `fund_token.move#L1520–L1529`

Recommendations

Short term, associate the multisig accounts with their underlying configuration (i.e., the list of owners and thresholds). For example, create a new struct representing such a configuration and associate it with the account through a `SmartTable`.

Long term, whenever a new feature is added, thoroughly document its intended behavior, define invariants, and use that information to drive testing and development.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Maturity Categories

The following tables describe the code maturity categories and rating criteria used in this document.

Code Maturity Categories	
Category	Description
Arithmetic	The proper use of mathematical operations and semantics
Auditing	The use of event auditing and logging to support monitoring
Authentication / Access Controls	The use of robust access controls to handle identification and authorization and to ensure safe interactions with the system
Complexity Management	The presence of clear structures designed to manage system complexity, including the separation of system logic into clearly defined functions
Cryptography and Key Management	The safe use of cryptographic primitives and functions, along with the presence of robust mechanisms for key generation and distribution
Decentralization	The presence of a decentralized governance structure for mitigating insider threats and managing risks posed by contract upgrades
Documentation	The presence of comprehensive and readable codebase documentation
Testing and Verification	The presence of robust testing procedures (e.g., unit tests, integration tests, and verification methods) and sufficient test coverage
Transaction Ordering	The system's resistance to transaction-ordering attacks

Rating Criteria	
Rating	Description
Strong	No issues were found, and the system exceeds industry standards.
Satisfactory	Minor issues were found, but the system is compliant with best practices.
Moderate	Some issues that may affect system safety were found.
Weak	Many issues that affect system safety were found.
Missing	A required component is missing, significantly affecting system safety.

Not Applicable	The category is not applicable to this review.
Not Considered	The category was not considered in this review.
Further Investigation Required	Further investigation is required to reach a meaningful conclusion.

C. Non-Security-Related Findings

The following recommendations are not associated with specific vulnerabilities. However, they enhance code readability and may prevent the introduction of vulnerabilities in the future.

- **Call `add_activity` in `process_settlements`.** The logic of the `add_activity` function is re-implemented in `process_settlements` (figure C.1).

```
1127 fun process_settlements(  
1128     account: address,  
1129     date: u64,  
1130     price: u64,  
1131     memo: String,  
1132 ) acquires ManagedShareHolders, FundManagement {  
[...]  
1185     let date = timestamp::now_seconds();  
1186     let params = format1(&b"price = {}", price);  
1187     let activity = Activity {  
1188         account,  
1189         tx_type: TX_TYPE_SETTLEMENT,  
1190         date,  
1191         memo,  
1192         tx_id: request_id,  
1193         params,  
1194     };  
1195     let counter = shareholder.activities_counter;  
1196     simple_map::add(&mut shareholder.activities, counter,  
activity);  
1197     shareholder.activities_counter = counter + 1;  
[...]  
1244 }
```

Figure C.1: `funds/sources/fund_token.move#L1127-L1244`

- **Remove all usages of the following parameters.** These parameters have no impact on system execution:
 - a. `ManagedShareHolders::shareholder_accounts_with_pending_trans actions`
 - b. `ManagedShareHolders::shareholder_accounts_with_pending_trans actions_counter`
 - c. `ManagedShareHolders::shareholder_accounts_counter`
 - d. `Shareholder::account`

- **Rename `ShareHolder::counter` to `ShareHolder::request_counter`.** Since there are more than one counters used within the `ShareHolder` struct, each counter should have a unique name that highlights its usage.
- **Removes all usages of the following functions.** The functions below all check whether a `u64` is greater than or equal to zero. Since the minimum value of a `u64` is zero, these functions are superfluous:
 - a. `fund_token::only_positive_amount`
 - b. `fund_token::only_valid_shares_value`
- **Rename `test_trasnfer_to_frozen_account_denied` to `test_transfer_to_frozen_account_decided` (figure C.2).**

```
271 fun test_trasnfer_to_frozen_account_denied(
```

Figure C.2: funds/sources/erc20_wrapper.move#L1127–L1244

- **The `request_id` is not unique.** It is derived from a counter that increases and decreases as pending requests are created and settled. This could lead to undesired behavior. For example, when a new pending transaction is created an event is emitted; however, this event will not be unique, as the same account could produce multiple requests with the same ID over time.

```
1087 aptos_framework::event::emit(CreateTransactionRequestEvent {
1088     account,
1089     tx_id: request_id
1090 });
1091 }
```

Figure C.3: funds/sources/fund_token.move#L1087–L1091

- **Use of `upsert` with non-unique IDs.** While this audit did not identify issues related to this, it is important to highlight that `upsert` will update the existing value if there is one.

```
1069 // add to pending transactions
1070 let request_id = managed.pending_transactions_counter;
1071 smart_table::upsert(&mut managed.pending_transactions, request_id, tx);
1072 managed.pending_transactions_counter = request_id + 1;
```

Figure C.4: funds/sources/fund_token.move#L1069–L1072

D. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

From September 3 to September 6, 2024, Trail of Bits reviewed the fixes and mitigations implemented by the Franklin Templeton team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, of the 12 issues described in this report, Franklin Templeton has resolved six issues, and has not resolved the remaining six issues. For additional information, please see the Detailed Fix Review Results below.

Please note that we were not provided fixes for issues 13–18, as these were identified as part of an **add-on best-effort review** of a few additional features.

ID	Title	Status
1	Users can request a share transfer when self-service is disabled	Resolved
2	Full liquidation requests will fail to execute	Resolved
3	A malicious shareholder can launch a DoS attack	Resolved
4	Shareholders can stop the admin from removing them as a shareholder	Unresolved
5	Insufficient event generation	Resolved
6	Frozen shareholders can be distributed dividends	Unresolved
7	Insufficient checks during request settlement	Unresolved
8	A high number of pending requests might lead to a temporary DoS	Unresolved

9	Pending requests are not settled in the order in which they were created	Resolved
10	The order in which the admin inputs the accounts to the settlement function can alter the outcome of the settlement	Unresolved
11	Creating requests just before the end of day allows shareholders to instantly settle them	Unresolved
12	The end_of_day function is broken	Resolved

Detailed Fix Review Results

TOB-FT-APTOS-1: Users can request a share transfer when self-service is disabled

Resolved in 57b245ed608. The Franklin Templeton team has updated the request_self_service_share_transfer function to ensure that it can be called only if self-service is enabled.

TOB-FT-APTOS-2: Full liquidation requests will fail to execute

Resolved in 96bc8c22209. The Franklin Templeton team has updated the request_full_liquidation function by passing the correct transaction type (TX_TYPE_FULL_LIQUIDATION) to the create_pending_transaction function.

TOB-FT-APTOS-3: A malicious shareholder can launch a DoS attack

Resolved in 18155da273f. The Franklin Templeton team has updated the cancel_self_service_request function to prevent a shareholder from deleting other shareholders' requests. Additionally, the function can now be called only if the request being deleted was created using self-service.

TOB-FT-APTOS-4: Shareholders can stop the admin from removing them as a shareholder

Unresolved.

The client provided the following context for this finding's fix status:

On hold until self-custody is implemented.

For now we are leaving the logic to limit the type and number of requests that a shareholder can generate on the client side. Especially since we are the only ones that can invoke this API in the first place. This includes self-service operations since we create the Aptos account on behalf of the user we retain control of the shareholder account for now. This could change in the future so we will adapt our business logic as needed.

TOB-FT-APTOS-5: Insufficient event generation

Resolved in d286c0ac33f. The Franklin Templeton team has updated the various modules to ensure that all state-changing functions emit events.

TOB-FT-APTOS-6: Frozen shareholders can be distributed dividends

Unresolved.

The client provided the following context for this finding's fix status:

Frozen shareholders are eligible for dividend distribution by design. We will make this behavior clear to the user in our documentation.

TOB-FT-APTOS-7: Insufficient checks during request settlement

Unresolved.

The client provided the following context for this finding's fix status:

On hold until self-custody is implemented.

For now we are leaving the logic to limit the type and number of requests that a shareholder can generate on the client side. Especially since we are the only ones that can invoke this API in the first place.

We as well provide the fund admin with a special APIs (i.e. `adjust_shares`, `cancel_request`, etc...) within the contract to deal with any discrepancies or adjustments needed to any shareholder balance or registration/freeze status.

TOB-FT-APTOS-8: A high number of pending requests might lead to a temporary DoS

Unresolved.

The client provided the following context for this finding's fix status:

On hold until self-custody is implemented

For now we are leaving the logic to limit the type and number of requests that a shareholder can generate on the client side. Especially since we are the only ones that can invoke this API in the first place. This includes self-service operations since we create the Aptos account on behalf of the user we retain control of the shareholder account for now. This could change in the future so we will adapt our business logic as needed.

TOB-FT-APTOS-9: Pending requests are not settled in the order in which they were created

Resolved in 22724f7e6ac. The Franklin Templeton team has updated the settlement processing logic to process pending requests in first-in-first-out order.

TOB-FT-APTOS-10: The order in which the admin inputs the accounts to the settlement function can alter the outcome of the settlement

Unresolved.

The client provided the following context for this finding's fix status:

The order of the accounts does not affect the outcome of the settlement for the entire fund. There is no advantage for any shareholder over another because our implementation is a 40 act fund in which the price for the day is only known at the time of the settlement. The price known to the shareholder at the moment of request creation is only a historic value for an initial estimation of the value of any shareholder balance. We also have controls in the client side (off-chain) to minimize discrepancies after settlement.

Even if discrepancies arise the contract API provided to the admin allows for adjustments.

TOB-FT-APTOS-11: Creating requests just before the end of day allows shareholders to instantly settle them

Unresolved.

The client provided the following context for this finding's fix status:

The time of transaction creation does not affect the outcome of the settlement for the entire fund. There is no advantage for any shareholder over another because our implementation is a 40 act fund in which the price for the day is only known at the time of the settlement. The price known to the shareholder at the moment of request creation is only a historic value for an initial estimation of the value of any shareholder balance. We also have controls in the client side (off-chain) to minimize discrepancies after settlement.

Even if discrepancies arise the contract API provided to the admin allows for adjustments.

TOB-FT-APTOS-12: The end_of_day function is broken

Resolved in f4d2ad20cca. The Franklin Templeton team has updated the end_of_day function to correctly pass the date input parameter to the process_settlements function.

E. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	Description
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.